Toward Type Inference for EO

a working note: how decoration and ⊥ already tell us the types

Objectionary

https://github.com/objectionary/eo

July 2026

Abstract

EO has no written-down types, but it wants a tool that catches type mistakes before a program runs. This note explains, in plain terms, what "type" should mean in EO, why the obvious idea (compare the *name* of the returned object) breaks the moment we use decoration, and what to use instead: a notion of type based on an object's *shape*. We then do three concrete things: write down the grammar of these shapes, give the shape (the "signature") of each built-in object, and describe — as plain steps — a checker that works out the shape of any expression and reports an error when the shapes do not fit. A worked example shows the checker accepting a correct program and rejecting two broken ones. None of this is new type theory; the pieces are all known and are pointed to in footnotes for the curious. The only EO-specific parts are how we treat decoration and the failure value ⊥.

What you need to know (the whole vocabulary)

Everything below uses just these words and this notation. If a footnote names a paper, it is optional background, not required reading.

Type inference the tool figures out the "kind" of every value by itself, because EO writes none down.[1]

Nominal vs. structural *nominal*: two objects have the same type only if they share a *name*. *Structural*: they match if they have the same *shape* — the same attributes. EO needs structural.[2]

Usable-as (subtyping) "an X can go wherever a Y is wanted." If `my-num` has every attribute `number` has, then `my-num` is usable as a `number`.

Notation $X \to Y$ means "give it an X, get a Y"; $A \mid B$ means "an A or a B" (as in TypeScript); $T?$ means "a T, or ⊥" (the failure value); $\{\, a : \ldots,\ b : \ldots \,\}$ is an object with attributes $a, b, \ldots$; "for any A" before a type means it works for every type (the tool picks which).

1 The problem: EO names objects, but we care about their shape

Every built-in EO object (an "atom") already declares the *name* of what it returns, after a `/`. These are real lines from `number.eo`:

```
[as-bytes] > number
  [x] > plus  /number      # returns a number
  [x] > times /number
  [x] > gt    /bool        # returns a bool
```

A name like `number` or `bool` is a *nominal* tag. It works until we decorate. Decoration is EO's "`@`" attribute: an object with an `@` behaves like whatever its `@` points to, because a call it does not answer itself is passed down to `@`. Consider:

```
[] > my-num       # its name is 'my-num'; it behaves like a number
  5 > @
```

`my-num` is named `my-num`, but calls like `plus` or `gt` fall through `@` to the `5` inside, so it *behaves* as a number. A check that compares the name `my-num` against the expected `number` wrongly says "no."

You might try to save the name-based idea by remembering the whole decoration *chain* (`my-num` then `number`) and checking whether `number` is in it. That survives the case above but not this one:

[1] Figuring out types from unannotated code is *type inference*; the foundational algorithm is L. Damas & R. Milner, *Principal type-schemes for functional programs*, POPL 1982.

[2] Treating a type as a shape (a set of attributes) rather than a name is standard; a classic reference is M. Abadi & L. Cardelli, *A Theory of Objects*, Springer 1996.

```
[] > my-num       # @ is 'seq *', whose name is 'seq' -- 'number' is nowhere
  seq * > @
    "Hello"
    5
```

Here `my-num` still behaves like a number, because `seq` runs its steps and hands back the last one (`5`). But knowing that needs the *meaning* of `seq`, not its name. So the two examples mark a line: the first is answerable by names, the second only by behaviour. Any name-based scheme handles the first and fails the second — so we have to describe behaviour, i.e. *shape*. This is also why EO was right to switch off its old runtime check that compared names: the real check needs the object's shape, and finding a shape at runtime means running the program. Shape is better worked out ahead of time, by inference.

2 What "type" should mean: a shape, and decoration means "usable-as"

Let a type describe what an object *does*: which attributes it answers, and with what. `number`'s type is the shape $\{\,$`plus` : `number` $\to$ `number`, `gt` : `number` $\to$ `bool`, $\ldots\,\}$. The one EO-specific rule is about `@`:

> **Decoration rule.** If an object has `@` pointing to something of shape S, then that object answers everything S answers (directly, or by falling through `@`). So it is *usable as* an S.

That is the whole trick: `my-num` (which decorates `5`) is usable as a `number` because its `@` is a number. The name `number` does not disappear — it just becomes a convenient *alias* for that shape, so error messages can say "number" instead of printing the full attribute list.

3 The type grammar (the shapes a type can take)

A type is one of the following. Each line is a shape, with a tiny example.

An unknown, written A, B, ... a placeholder the tool has not filled in yet. A fresh unknown is created for each free attribute of a formation and gets pinned down as the tool learns more.

An object shape $\{\,\ell_1 : \tau_1,\ \ldots,\ \ell_n : \tau_n\,\}$ a list of attribute names with their types. One attribute may be the special `@`. A trailing "..." means "and possibly more attributes" — needed so a shape can stand for "at least these."[3]

A tuple, `tuple` A **or** $\langle A, B, C\rangle$ an ordered collection; either "a tuple of A's" or a fixed-length one with a type per position.

Failure, $\bot$, and the option $T?$ $\bot$ is the value a terminated computation produces; $T?$ means "a T, or $\bot$."

Either/or, $A \mid B$ a value that is an A or a B (for example, a branch that returns one of two things).

A self-referring shape, "loop $A.\ \ldots$**"** for objects that mention themselves, like a list whose tail is another list of the same kind.[4]

For the record (skippable), the same grammar in symbols:

$$\tau \ ::=\ A \ \mid\ \{\ell_1{:}\tau_1, \ldots, \ell_n{:}\tau_n\} \ \mid\ \texttt{tuple}\,\tau \ \mid\ \langle\tau, \ldots, \tau\rangle \ \mid\ \bot \ \mid\ \tau? \ \mid\ \tau \mid \tau \ \mid\ \mu A.\,\tau$$

4 Failure as a type: $\bot$ and `recovered`

EO recently made every unrecoverable failure produce one special value, $\bot$, and the *only* way to catch it is the object `recovered` (`recovered.eo`):

```
[value alternative] > recovered /bytes      # value, unless it is a bottom,
recovered (2.as-i64.div 0.as-i64) 7         # in which case: alternative  ==> 7
```

In types, this is simple and powerful. An operation that might fail has type $T?$ ("a T, or $\bot$"). Calling a normal attribute on a $T?$ is an error — you must clear the $\bot$ first. `recovered` is what clears it: give it a maybe-failed value and a fallback, and it hands back a definite value. Its type is

$$\texttt{recovered}:\ \text{for any } A, B:\quad A? \to B \to (A \mid B),$$

read as: "take an A-or-$\bot$ and a B; give back an A or a B" (the $\bot$ is gone). This is the same shape as "get the value out of a box, or use a default" in other languages.[5]

[3] The "at least these" part is *row polymorphism*: M. Wand, LICS 1987; D. Rémy, 1994.

[4] *Recursive types*; R. Amadio & L. Cardelli, 1993.

[5] This need to combine "a value or nothing" with "an A or a B" in one inferred type is exactly what *algebraic subtyping* was built to do well: S. Dolan, PhD thesis, Cambridge 2017; and a very readable version, L. Parreaux, *The Simple Essence of Algebraic Subtyping*, ICFP 2020 (about 500 lines of code).

Two doors, and safe chaining with `?.` Since a normal dispatch on a $T?$ is rejected, the programmer has exactly two ways forward, and the checker enforces the choice: *resolve* the failure now with `recovered`, or *defer* it with the fragile dispatch `?.`. The rule for `x?.y` is to look past a possible $\bot$, dispatch `.y` on what is inside (fully shape-checked), and re-wrap the answer as maybe-$\bot$ — so fragility rides along the chain, and `x?.y?.z?.a` comes out as $A?$, a value that is still maybe-$\bot$ until some `recovered` ends it. This is ordinary optional chaining (Rust's `?`, TypeScript's `?.`); it turns "I carried the $\bot$ this far" into something the compiler tracks and, at the first use as a definite value, forces you to settle. It does not switch off checking: `x?.y?.z?.bogus` is still rejected when that shape has no `bogus`. And `recovered` is checked too — `(recovered x alt).y` is accepted only when the fallback `alt` can also answer `.y`, so the two arms really are interchangeable.

Where $\bot$ comes from. The checker never *guesses* fragility from how code is written. $T?$ enters only from declared *sources* — an atom whose stated result is $T?$ (a lookup that can miss, a slice that can run off the end), the `?.` operator, and the rule just below — and then propagates by ordinary flow. An object is fragile in a given place exactly when its inferred type there is $T?$, traced back to one of those sources. (A corollary worth stating: today's parser output predates the $\bot$/`?.` syntax, so the real objects show almost no fragility yet — the machinery is exercised by hand-built cases and by the rule below.)

Incompleteness as fragility. EO adds one more source of $\bot$, and a tidy one. Some voids are optional recovery callbacks — `cant-convert` on a numeric conversion, `cant-connect` on a socket — which a confident programmer may leave unset. But an object with any attribute unset is *incomplete*, and an incomplete object is fragile: using it must go through `?.` or be recovered. This needs no new machinery — a formation that escapes as a value with voids still unfilled is given type $T?$; application is allowed to see through the ? (so `foo 1 2` still completes a two-argument `foo` to a definite value), while any plain dispatch on it meets the discipline above. Run over the whole runtime with the rule on, the checker flags exactly five objects: `number` and `i64` (a conversion used with `cant-convert` unset), `fs/file` (`cant-check`), `nk/socket` (`cant-receive`), and `fs/path` (an unapplied `uri`). Four are precisely the recovery-callback situations the rule is meant to catch. The fifth, reading `posix.separator`, is *not* a false positive. The rule is deliberately *object-level* — an object with any attribute unset is fragile, full stop — so dispatching `.separator` on a `posix` whose `uri` was never supplied is correctly rejected, even though `separator` reads only a constant. Reaching into a half-built object is exactly the smell the rule exists to forbid; it is not object-oriented, and the fix belongs in the EO code (give `separator` a home that does not demand a `uri`), not in a weaker, attribute-level rule. This is the same kind of latent-smell surfacing as the `while` and `switch` results below — a type the author never intended, pointing at a design that should change.

5 The atoms, annotated

The built-in objects are the *starting facts*: the checker cannot look inside them (they are implemented in Java), so we state their shapes by hand, and everything else is worked out from these. Writing these down is step one of any real effort — if a shape cannot be stated here, the grammar of Section 3 is not yet good enough. The key ones:

EO	Type	In plain words
`number.plus`	`number` $\to$ `number`	a number in, a number out
`number.gt`	`number` $\to$ `bool`	compare; give `true`/`false`
`bool.if`	for any A, B: $A \to B \to (A \mid B)$	pick one of two branches
`recovered`	for any A, B: $A? \to B \to (A \mid B)$	the value, unless $\bot$, then the fallback
`seq`	for any A: (tuple ending in A) $\to (A \mid$ `bool`$)$	run every step, give the last
`tuple.head`	for a `tuple` A: A	the last element (EO tuples grow at the head)
`tuple.with`	for a `tuple` A: $B \to$ `tuple` $(A \mid B)$	append one element
`map.found(k).get`	for any B: $B \to (V \mid B)$	the stored value, or your fallback

Two things to notice. `bool.if` shows why we need "for any A, B": the branches may have different types, and the answer is "one or the other." `seq`'s answer depends on *which* element is last, but that is fixed the moment the tuple is written, so no program needs to run — the tool just reads off the last position's type.

6 The checker, over the `@`-core

The core of EO has only four moves: **make** an object (a formation), **apply** arguments to it, **dispatch** an attribute (a call `e.m`), and **decorate** via `@`. A checker only has to handle these four, plus looking up names. Here it is

as plain steps; `typeof` returns the shape of an expression, and `usable-as` answers "can an X go where a Y is wanted?"

```
typeof(expr, env):                          # env maps names -> known types
  NAME  x:            return env[x]
  MAKE  [p1 p2 ...] with body:
      shape = {}
      for each free attribute p:            shape[p] = a fresh unknown
      for each named child  n = e:           shape[n] = typeof(e, env + params)
      if there is an '@' child e:             shape["@"] = typeof(e, env + params)
      return shape
  APPLY  f arg  (fills f's next free attribute p):
      ft = typeof(f, env);  at = typeof(arg, env)
      require  usable-as(at, ft.expected(p))          # else ERROR
      return  ft  with p replaced by at
  DISPATCH  e.m:
      t = typeof(e, env)
      if t is an option (T?):  ERROR "'m' used on a value that can be bottom;
                                      recover it first"
      look up m in t;  if missing, follow t["@"] and look again (repeat)
      if still missing:        ERROR "'m' is not an attribute of this object"
      return the found attribute's type

usable-as(X, Y):                            # "an X can go where a Y is wanted"
  if Y is an unknown:   set Y := X;  ok
  if X is an option and Y is not:   NOT ok           # a bottom might leak
  if both are object shapes:   ok  iff  every attribute Y requires is present
        in X (directly or through @), and usable-as holds on each of them
```

A worked run. Take a correct program:

```
[] > my-num
  5 > @
my-num.plus 1
```

1. `typeof(5)` is `number` (a number literal).
2. `typeof(my-num)`: a formation with an `@` child, so its shape is { @ : `number` }.
3. `my-num.plus` is a DISPATCH: `my-num` has no `plus` of its own, so follow `@` to `number`, which has `plus` : `number` → `number`. Found.
4. APPLY to `1`: `1` is a `number`, and `plus` wants a `number` — `usable-as` holds. Result: `number`.

Accepted; the whole thing is a `number`. Now two programs it rejects:

```
[] > my-num              (m.get "k").plus 1     # m.get can fail, so its type
  "hi" > @                                      # is 'number?' (number or bottom)
my-num.plus 1
```

On the left, `typeof(my-num)` is {@ : `string`}; DISPATCH of `plus` finds none on `my-num`, follows `@` to `string`, finds none there either — *error:* "`.plus` is not an attribute of `my-num` (it decorates a string)." On the right, the receiver has type `number?`, so DISPATCH stops immediately — *error:* "`.plus` used on a value that can be ⊥ (from `m.get`); recover it first, e.g. `recovered (m.get "k") 0`." Recovering turns the `number?` back into a `number` (that is `recovered`'s type), and the fixed program is accepted. Both mistakes are caught without running anything.

7 The spine: solving, and surviving recursion

The steps in Section 6 decide each fit *on the spot*. Two things break that — the top two gaps from typing real objects by hand — and both are fixed by one idea used twice: **assume, then reconcile**.

Collect, then solve. Split typing into two passes. *Collect:* walk the code once, give every unknown a fresh name, and instead of deciding fits, just *write down* each requirement as a note — "this unknown must be usable as a `number`," "this one must have a `.head`," "this value flows into that slot." *Solve:* work through the notes and pin each variable down. Why two passes? A variable often makes sense only once you have seen all its uses: if `x`

is used once where a number is wanted and once where a string is wanted, no single on-the-spot decision is right, but the two notes together say `x` is `number | string`. Deciding early cannot produce that union; collecting can.

Solving needs almost no machinery. Give each variable two lists: **flows-in** (things put into it) and **flows-out** (slots it is used in). To process a note "X usable-as Y": if X is a variable, add Y to its flows-out; if Y is a variable, add X to its flows-in; either way, re-check that everything already on the other list still fits (which may make smaller notes); if both are object shapes, break it into one note per attribute Y asks for; and if X can be $\bot$ while Y cannot, that is the "recover it first" error. A variable's final type is *everything that flowed into it, or-ed together*. So unions and "for any" are never invented by hand — they fall out of the bookkeeping.[6]

Tie the knot. Recursion — an object that mentions itself — made the Section 6 walk loop forever (`seq`, `map`, `switch`, `while`, and `number` all do it). Same idea, on the object: before looking *inside*, give it a fresh variable `t` and remember "this object = `t`." Now type the body; every self-reference finds `t` — a name, not another trip inside — so the walk stops. The body works out to a shape S; close the loop by recording `t` = S. Since S may contain `t`, that is a self-referring shape — the "loop $A.\ldots$" from Section 3. One last loop hides in the solver: two self-referring types can bounce a note back and forth forever, so the solver keeps a **seen** set and stops when a note comes back. Assume-then-reconcile, one level down.

Here is Section 6's checker with the spine in place — `infer` returns a type (with variables) and piles up notes; `solve` settles them:

```
infer(expr, env):
  NAME x:   return env[x]
  MAKE  [free...] with children and maybe an '@':
      t = fresh variable
      env2 = env + (this object -> t)       # register BEFORE descending
      for each free attribute p:  shape[p] = fresh variable
      for each child  n = e:      shape[n] = infer(e, env2 + params)
      if there is an '@' child e:  shape["@"] = infer(e, env2 + params)
      note( t = shape )                     # tie the knot
      return t
  APPLY  f arg:
      r = fresh variable
      note( infer(f,env) usable-as (takes infer(arg,env), gives r) )
      return r
  DISPATCH  e.m:
      r = fresh variable
      note( infer(e,env) usable-as { m : r, ... } )
      return r

solve(note "X usable-as Y", seen):
  if (X,Y) in seen: return                  # stop cycles (recursive types)
  seen.add (X,Y)
  if X is a variable:  X.flowsOut += Y;  for L in X.flowsIn:  solve(L usable-as Y)
  elif Y is a variable: Y.flowsIn += X;  for U in Y.flowsOut: solve(X usable-as U)
  elif both are shapes: for each label m in Y:  solve(X[m] usable-as Y[m])
  elif X can be bottom and Y cannot:  ERROR "recover the bottom first"
# afterwards: each variable's type = the union of its flows-in
```

A recursive run that used to loop. Take a small self-referring object:

```
[tup] > last
  if. > @
    tup.tail.length.eq 0
    tup.head
    last tup.tail
```

1. Start on `last`: fresh `t`, remember `last` = `t`.

[6] This two-list flows-in/flows-out bookkeeping is essentially Simple-sub, the readable form of algebraic subtyping from the Section 4 footnote — about 500 lines including a parser.

2. Walk the body. `tup` is a fresh `u`. `tup.head` notes "`u` has `head` of type `w`"; `tup.tail` notes "`u` has `tail` of type `v`."
3. `last tup.tail` is the self-call. `last` is `t` — a *variable* (from step 1), *not* another descent — so the walk **stops here**, noting "`t` takes `v`, gives `r`."
4. `if. cond A B` gives `A | B`, here `w | r`. Close the knot: `t` = (`u` → (`w` | `r`)), self-referring through `r`.
5. Solving settles the notes: `u` is a tuple, and `w`, `r` are its element type, giving `last` the type "for any A: `tuple` $A \to A$". No loop, no annotations. (That "`tail` of a tuple is the same tuple" step needs the tuple rules — the next piece to add.)

The spine closes gaps 1 and 2. The remaining Tier-1 items — tuple rules, applying the "for any" atom signatures, and pushing types into callbacks — build directly on this collect-and-solve frame.

8 Completing the core: tuples, generic atoms, callbacks

With the spine (Section 7) in place, the other three Tier-1 gaps are each a small addition to the same collect-and-solve frame.

Tuples. Give a tuple two possible shapes: a *fixed* one, $\langle \tau_1, \ldots, \tau_n \rangle$, when the length is known (as in `* a b c`), and a *loose* one, `tuple` A ("every element is usable as A"), when it is not (as when a tuple is grown in a loop). The accessors follow — recalling that EO tuples grow at the head, so `head` is the *last* element:

- `* a b c` : $\langle \tau_a, \tau_b, \tau_c \rangle$; `t.length` : `number`.
- `t.head` : τ_n (fixed) or A (loose); `t.tail` : $\langle \tau_1, \ldots, \tau_{n-1} \rangle$ or `tuple` A.
- `t.with x` : $\langle \tau_1, \ldots, \tau_n, \tau_x \rangle$ or `tuple` $(A \mid \tau_x)$.

The solver already breaks a shape into one note per part, and a tuple is just another shape: "X usable-as $\langle \ldots \rangle$" becomes one note per position (fixed) or one note on the shared element (loose). The fixed form is what lets `seq` and `switch` pin their result to the *last* element without running anything; a loose tuple falls back to the union of its elements.

Generic atoms: instantiate on use, generalize on definition. The Section 5 signatures are *schemes* — types with "for any" holes — and two moves connect them to the checker. **Instantiate:** at each use, make *fresh* copies of the "for any" variables, then note the arguments against them. `recovered 42 7` makes a fresh `A` and `B`, notes "`42` flows into `A`?" (so `A` becomes `number`) and "`7` flows into `B`" (`B` becomes `number`), and the result `A | B` solves to `number` — an argument-dependent result computed by the solver, no evaluation, and the fresh copies stop two calls at different types from tangling. **Generalize:** when a user object's inferred type still has unknowns that nothing outside constrains, freeze them as "for any," turning a one-off type into a reusable scheme so `reduced` works at every element type. These are the two standard moves of Hindley–Milner, with each variable's flows-in/flows-out bounds riding along.

Making it sound: levels. One question decides whether generalization helps or hurts: *which* unknowns may be frozen as "for any"? Freeze one that is really shared with the surrounding object and you duplicate it wrongly; freeze too few and a reused object collapses all its call sites into one — the path object `fs/path` uses its helper `posix` at a dozen sites, and without generalization their arguments merge, so a parameter that is a string at every real call is inferred as an impossible `string` & `bytes` and a correct object is rejected. The standard cure is to stamp every unknown with a **level** — the nesting depth of the object being defined — and, at a use, to instantiate exactly the variables *deeper* than that use while sharing the rest; a companion step, *extrusion*, lowers the level of any variable that escapes to a shallower scope so that nothing still-shared is ever frozen. Levels are what make "instantiate on use, generalize on definition" sound rather than merely plausible, and they are the part the prototype in the next section spent the most care getting right.

Callbacks: push the expected type in. An atom that takes a function says so in its signature — `malloc.for`'s scope is `block -> R`, `while`'s condition is `number -> bool`. When the argument is a formation `[m] ...`, do not type it blind: *check* it against that expected function type, which seeds the parameter (`m` is a `block`) before typing the body — so `m.put` and `m.read` resolve, and any mistake is reported right at the callback. Having the checker both *infer* a type and *check against* an expected one is the only real addition

here. One rule makes it work: to fit one function where another is wanted, the results go the same way but the *arguments go the opposite way* — a `block -> R` is wanted, so the callback's parameter must be willing to *accept* a `block`. That backwards flow is what carries the `block` into `m`, and it is the one genuinely counter-intuitive rule in the whole system.[7]

With the spine and these three additions, the checker can walk the whole standard library — which is what the next section shows.

9 It scales to real code: a working checker

These rules are not only on paper. A prototype of about 900 lines implements exactly this — the grammar, the annotated atoms, the collect-then-solve spine, and the level-based generalization above — and reads EO's *real* parser output (the XMIR under `eo-runtime/target/eo/1-parse`). Run over the whole runtime it types **all 117 top-level objects**, with no rejections, in under a second. And — the actual test of a type system — it still *catches* mistakes: a string sent `.plus`, a value used at once as a function and as a string, a dispatch on an attribute that is not there are all reported, and every deliberately-wrong example in Sections 4–8 rejects as designed. Passing everything is trivial for a checker that says yes to everything; this one says no in the right places.

The standard library comes out as the rules predict. The list combinators (`tuple/mapped.eo`, `tuple/reduced.eo`) are generic: `mapped` has type "for any A, B: `tuple` $A \to (A \to B) \to$ `tuple` B" and `reduced` is the usual fold. Their callbacks carry no annotations — the tool reads each callback's type from its body — so `mapped (* 1 2 3) (x.times 2 > [x])` is a tuple of numbers, while a callback returning a `bool` makes a later `.plus` on an element a caught error. A `map` never returns a bare value: `found(key).get` *demands* a fallback, so a lookup has type "value *or* fallback" — absence lives in the type. And `switch` returns `true` when no case matches, so its inferred type carries a stray `bool` the author never intended: the type quietly points at a real bug no test triggers as long as some case matches.

What took the care. Almost all of it was the level machinery, and four subtleties were decisive — each a place the obvious implementation is wrong. The self-reference that ties a recursive knot must live *inside* the object's own scope, not outside it, or generalization silently de-generalizes the parameters and the checker stops catching mistakes. The built-in atoms must sit at the outermost level, or every use of one drags the solver into needless extrusion and it crawls. The parent link ρ must be left out when measuring an object's level, or a deep parent keeps a record "too deep" to extrude and solving never stops. And a large shared type must be printed by naming each variable once, or `fs/path`'s type — a wide, cyclic shape — prints exponentially. None of these touches the design of Sections 3–8; all are the ordinary hazards of a level-based solver, now charted.

10 Is this doable, and your "just compile it" idea

Yes, and it is not exotic: a type based on shapes, worked out automatically, with "usable-as" and "A or B," is a well-charted design.[8] It belongs in a compile-time step over XMIR (EO's intermediate form), never in the runtime.

Your idea of translating EO into a strictly-typed language (say Haskell) and letting its compiler judge is a real technique, and half of it fits EO perfectly: $\bot$ maps to Haskell's `Maybe` and `recovered` to `fromMaybe`. The catch is the other half. Haskell decides types by *name* and has no built-in "usable-as," which is precisely EO's decoration — so you would rebuild the hard part inside Haskell's most awkward corner, the errors would come out talking about the generated Haskell rather than the user's EO, and "if it compiles, it is safe" is only true if the translation is faithful, which you can only know by writing down the rules in Sections 3–8 first. If you do want to borrow a compiler, pick a *structurally*-typed one: TypeScript already decides types by shape and has $A \mid B$ and "value or nothing" built in — almost the whole EO picture — and OCaml's object types are the same idea with more rigour. Best use: a quick prototype to test these rules, not the finished checker. Either way the rules come first.

So the order is: (1) this grammar; (2) these atom shapes; (3) this checker, first on the four-move core, then widened, dog-fooded on the standard library. The parts that will take real care are self-referring shapes (the "loop" case), keeping the tool's guesses predictable, and error messages that speak EO. None is unexplored; all

[7] Both *inferring* and *checking against* an expected type is *bidirectional typing*: a friendly survey is J. Dunfield & N. Krishnaswami, *Bidirectional typing*, ACM Computing Surveys, 2021. "Arguments go the opposite way" is standard function subtyping — contravariant in the argument, covariant in the result.

[8] The combination — structural shapes, "usable-as," unions, and full inference — is the subject of the algebraic-subtyping work cited in Section 4; a gentle textbook is B. Pierce, *Types and Programming Languages*, MIT Press 2002.

are documented. EO is ready to start — the hard idea, that decoration forces us to talk about shape, is already behind us.[9]

[9]EO and its calculus: Y. Bugayenko, *EOLANG and φ-calculus*, arXiv:2111.13384, 2021; N. Kudasov & V. Sim, *Formalizing φ-calculus*, arXiv:2204.07454, 2022.